

Week 2 Day 3: Buildable Image Delivery and Failure Analysis

Overview

Day 3는 Dockerfile 문법만 외우거나 명령어만 따라 치는 날이 아니다. 제공된 정적 앱을 **다른 사람이 같은 결과로 실행할 수 있는 image artifact**로 포장하고, build/run/verify 중 어디에서 실패했는지 증거로 좁히는 날이다.

오늘의 핵심 질문은 다음과 같다.

내 앱을 image로 납품하려면 Dockerfile, build context, layer/cache, tag/digest, registry gate, run/verify/security scan 기준이 어떻게 남아야 하는가?

Concept Map

개념	반드시 설명할 내용	실습에서 확인할 증거
Dockerfile	image를 만드는 build recipe이자 실행 계약서다. base image, 복사할 파일, 시작 process, container port를 정의한다.	Dockerfile, docker image inspect
Build	source와 Dockerfile을 build context로 보내 image artifact를 만든다. build 성공은 아직 서비스 정상 확인이 아니다.	docker build -t, docker images
Layer	Dockerfile instruction이 누적되어 image 변경 이력을 만든다. layer를 보면 무엇이 image에 들어갔는지 추적할 수 있다.	docker history
Cache	이전 build layer를 재사용해 rebuild를 빠르게 만든다. 어떤 instruction 뒤에서 cache가 깨졌는지 보면 변경 영향이 보인다.	build output의 CACHED, rebuild 시간/출력
Tag	사람이 읽기 쉬운 image 이름표다. 새 build가 아니라 기존 image에 reference를 추가할 수도 있다.	docker tag, docker images
Digest	registry에서 content를 식별하는 값이다. tag보다 재현성 판단에 강하다. local image에서는 digest가 비어 있을 수 있다.	RepoDigests
Registry	image를 다른 machine/runner/Kubernetes가 pull할 수 있게 저장하는 장소다. push 전 secret/public 범위 gate가 필요하다.	Docker Hub/ECR push gate, RepoTags
Run & verify	image를 container로 실행하고 사용자가 접근할 정상 기준을 확인한다. Up과 HTTP 200은 다른 증거다.	docker run, docker ps, curl
Vulnerability scan	image 안의 OS/package 취약점이 있는지 확인한다. scan 결과는 배포 차단/보류/예외 판단의 근거다.	docker scout cves, scan note

Learning Goals

- Dockerfile이 어떤 실행 계약을 image에 담는지 설명한다.
- build context와 .dockerignore가 secret risk, build speed, image size에 미치는 영향을 확인한다.
- 표준 앱을 build/run하고 HTTP 200과 vulnerability scan 결과로 정상/안전 기준을 남긴다.
- layer/cache를 보고 변경 영향과 rebuild 이유를 설명한다.
- tag, digest, registry, optional push gate를 release handoff 기준으로 설명한다.
- missing file, wrong CMD, wrong port, bloated context를 build/run/verify 단계로 분류한다.

Lesson Index

- 1교시: Delivery target - image, container, artifact, run/verify 기준 잡기
- 2교시: Dockerfile contract - Dockerfile instruction을 실행 계약으로 설명하기
- 3교시: Build context gate - build context와 .dockerignore 위험 설명하기
- 4교시: Build/run/verify/scan - 실행 가능하고 기본 취약점 점검을 거친 이미지 후보 만들기
- 5교시: Layer/cache evidence - layer와 cache로 변경 영향 설명하기
- 6교시: Failure drill - build/run/verify 실패를 RCA로 분류하기
- 7교시: Tag/digest/registry gate - image 식별과 공유 기준 설명하기
- 8교시: Delivery handoff - README에 build/run/check/cleanup과 실패 기준 남기기

Practice Files And Assets

자료	용도
assets/day3-image-build-registry-overview.png	Day 3 image/build/registry 전체 구조 인포그래픽
assets/lesson-08-delivery-handoff-table.png	8교시 image delivery 인수인계 표 인포그래픽
labs/static-site/index.html	표준 실습 앱 HTML
labs/static-site/styles.css	표준 실습 앱 CSS
labs/static-site/Dockerfile	Dockerfile build 실습
labs/static-site/.dockerignore	build context 제외 규칙
labs/static-site/README.md	build/run/check/cleanup 예시
hands-on-lab.md	Day 3 전체 실행 흐름

Instructor Rule

명령어를 던지고 넘어가지 않는다. 각 명령 전에는 무엇을 확인하려는가, 명령 후에는 어떤 출력이면 성공인가, 실패하면 어느 단계 문제인가를 말한다. 중복 설명은 제거하되, Dockerfile/build/layer/cache/tag/digest/registry/run/verify의 첫 등장에는 반드시 개념 설명을 둔다.

Common Rules

- docker build의 마지막 .은 build context다. 명령 위치와 포함 파일을 먼저 확인한다.
- .env, token, 개인 파일, log, dependency directory는 image/context에 들어가지 않게 .dockerignore로 막는다.
- EXPOSE는 문서화된 container port일 뿐 host port publish가 아니다. host 접근은 docker run -p host:container로 확인한다.
- latest는 재현성 기준이 아니다. 수업에서는 명시적 tag를 쓰고, provenance 판단에는 digest/official image 여부를 함께 본다.
- container cleanup와 image cleanup은 다르다. image 삭제는 다음 실행을 다시 build/pull하게 만든다.

Completion Definition

1. 표준 앱 image를 build했다.
2. container run, HTTP 200, vulnerability scan 결과로 acceptance check를 남겼다.
3. Dockerfile instruction과 build context 경계를 설명했다.
4. history/inspect/cache 중 2개 이상을 evidence로 확인했다.
5. build/run/verify failure 중 하나를 RCA로 분류했다.
6. tag/digest/registry/push gate와 취약점 scan 판단을 README 또는 handoff note에 남겼다.

Next Connection

Day 4는 Day 3에서 만든 image를 여러 runtime config와 failure 조건으로 실행한다. Day 3가 image artifact를 납품하는 날이라면, Day 4는 그 artifact가 다른 runtime 조건에서 왜 정상/장애가 되는지 logs, inspect, exec, stats로 분석하는 날이다.